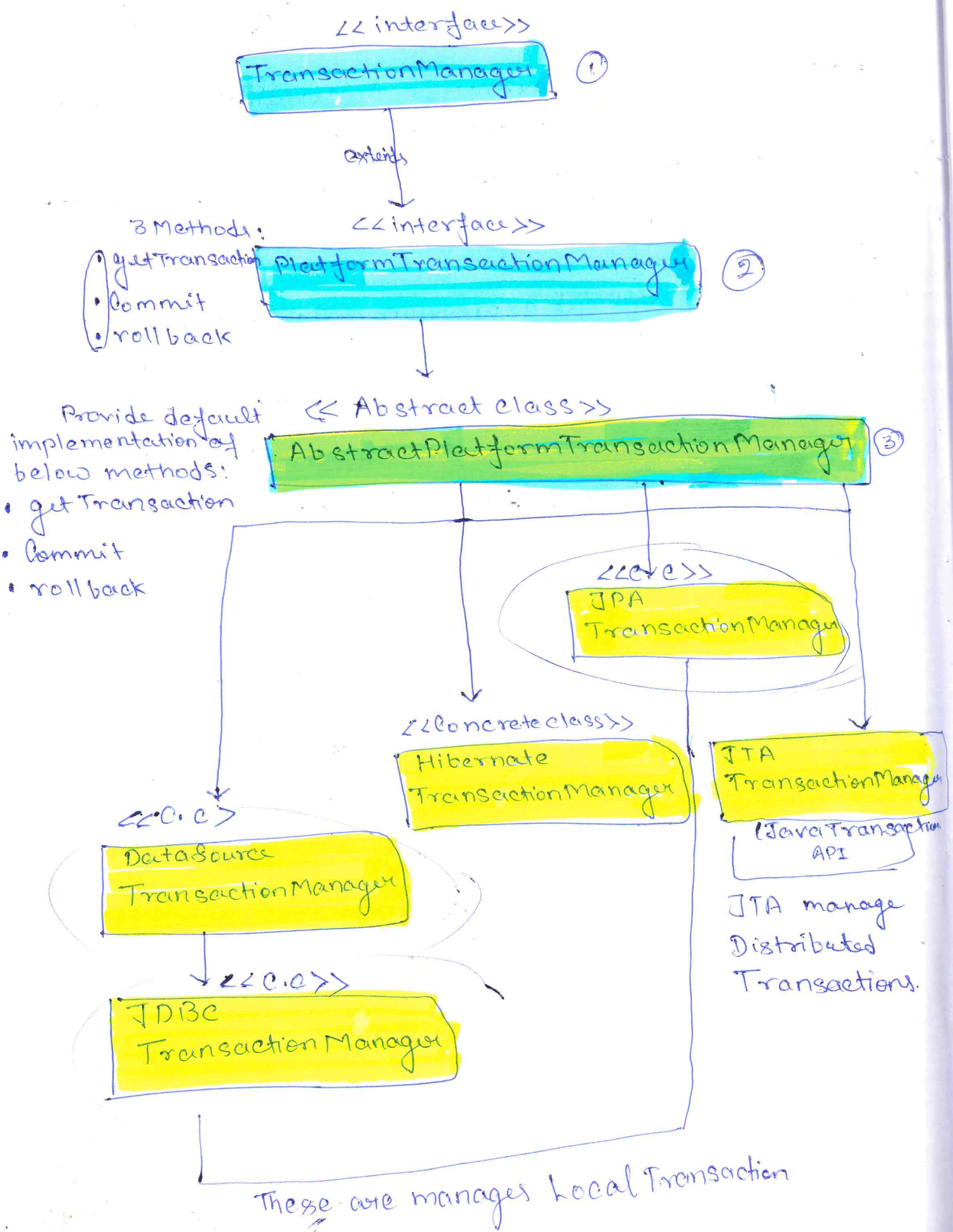
Lecture-13

Hierarchy of Transaction Manager



Concept && Coding

Transaction Management

~~Det~~ Declarative

Transaction Management through Annotation.

```
@Component
public class User{

  @Transaction
   Public void updateUser(){
        S.O.P("Update QueryTo update the user db value");
  }
}
```

* Here based on undeling DataSource spring boot will chose appropriate Transaction manager.

```
@Configuration
Public class AppCofig{

  @Bean
   Public DataSource dataSource(){
    DriverManagerDataSoure datasoura = new D.M.D.S();
          setDriverclassName("org.h2.Drive");
          Set Url ("jdbc:h2:mem:testdb");
          set usernam("sa")
           set Password("");
        return datasource;
  }
  @Bean
  Public PlatformTransactionManager userTransact(Datasou
                                                  datas
    return new DataSourceTransactionManager(datasource);
```

Concept && Coding

Programmatic

Transaction Management through code
Flexible but difficult to maintain

Why Flexible? Let consider with example

```
@Component
Public class User{

    @Transactional
      Public void updateUser(){
        //1: Update DD
        //2: External API Call
        //3: Update DB
    }
}
```

Approach 1

App Config class

```
@Component
Public class UserProgramonatic-Approach1{
    @Autowired
    PlatformTransactionManager userTransactionManager;

    Public void updateUseProgrammatic (){
        TrasactionStatus status = UserTransactionManager.getTransaction;
        try{
            // Some intitial set of DB Operation.
            userTransactionManager.comuit(status);
        }
        catch (Exception e){
            userTransaction.rollback(status);
        }
```

Concept && Coding

Approach 2

App Config

```
Public TransactionTemplate transactionTemplate(PlatformTransactionManager manager)
{
return new TransactionTemplate(manager);
}
```

```
@Component
Public class UserProgrammaticApproach2 {
    @Autowired
    Transaction transactiontemplate;

    Public void updatUser Programmatic() {
    TransactionCallback<TransactionStatus> dboperation = (T.Status) ->{
            S.o.p ("Insert Query ran");
            S.o.p ("Update Query ran");
        return status;
    };
    TransactionStatus status = transactionTemplate.execute(dboperation);
    }

}
```

Concept && Coding

Propagation and It's Type in Transaction

- When we try to create a new Transaction, it first check the PROPAGATION value set, and this tell whether we have to create new transaction or not.

1:- ~~REQUIRED (default propagation)~~:

```
@Tranasaction
method1()
{
    ————
    ————
    ————
    method2();
}

getTransaction(method1)
```

```
@Transaction
method2(){
    ————
    ————
    ————
}
```

getTransaction(metho2)

```
Txn1
{

}
Txn2
{

}
```

Concept && Coding

1:- RIQUIRED (default propagation):

```
@Transaction (propagation = Propagation.REQUIRED)
  if (parent txn present)
      Use it ;
  else
      Create new transaction.
```

```
Txn1 {  method 1

~~Txn1~~
}
        @Txn
        method 2
        (    )
```

2:- REQUIRED_NEW :-

```
@Transactional (propagation = Propagation.RIQUIRED_N
  if (parent txn present)
      Suspend the parent txn;
      Create a new Txn and once finished;
      Resume The parent txn;
  else
      Create new transaction and execute the method;
```

Concept && Coding

3:- **SUPPORTS:-**

```
@Transactional(propagation = Propagation.SUPPORTS)
  if (parent txn present)
      Use it;
  Else
    Execute the method without any transaction.
```

: **NOT_SUPPORTED:-**

```
@Transactional(propagation = Propagation.NOT_SUPPORTED)
  if (parent txn present)
    Suspend the parent txn;
    Execute the method without any transaction;
    Resume the parent txn;
  else
    Execute the method without any transaction;
```

:- **MANDATORY:**

```
@Transactional(propagation = Propagation.MANDATORY)
  if (parent txn present)
      Use it;
  Else
    Throw Exception
```

Concept && Coding

- NEVER

```
@Transactional(propagation = Propagation.NEVER)
    if (parent txn present)
        Throw exception;
    Else
        Execute the method without any transaction;
```

Declarative way of usage :-

Concept && Coding